

INSTITUTO FEDERAL DE MINAS GERAIS (IFMG)
CAMPUS BAMBUÍ
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Disciplina: BiSuEEA. 512 – Sistemas Embarcados
Professor: Williams L. Nicomedes

Henrique Augusto G. Fernandes

**LISTA DE EXERCÍCIOS N°5 – PROJETO DE CIRCUITOS COMBINACIONAIS COM
PROCESSOS**

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	Fundamentos Teóricos	4
2.1.1	<i>Processos em VHDL</i>	4
2.1.2	<i>Estruturas de Controle Sequenciais</i>	4
2.2	Identificação das Variáveis de Entrada e Saída	4
2.2.1	<i>Variáveis de Entrada</i>	4
2.2.2	<i>Variáveis de Saída</i>	5
2.3	Tabela-Verdade	5
2.4	Código VHDL	5
2.4.1	<i>Estrutura do Código</i>	6
2.4.2	<i>Justificativa do Uso de Processos</i>	7
2.4.3	<i>Cobertura Completa do Case</i>	7
2.5	Implementação no Quartus II	7
2.5.1	<i>Geração do Bloco Lógico</i>	7
2.5.2	<i>Diagrama Esquemático</i>	8
2.6	Simulação e Validação	8
2.6.1	<i>Configuração das Formas de Onda</i>	8
2.6.2	<i>Resultados da Simulação</i>	9
2.6.3	<i>Análise dos Resultados</i>	9
3	CONCLUSÃO	11

1 INTRODUÇÃO

Este documento apresenta a resolução da Lista de Exercícios nº5 da disciplina de Sistemas Embarcados, referente ao 2º semestre de 2025. O objetivo consiste em projetar um decodificador 3 para 8 com entrada de habilitação (*enable*) utilizando a linguagem de descrição de hardware VHDL.

Conforme apresentado nas Notas de Aula 16, um decodificador é um circuito combinacional que recebe um código binário de N bits na entrada e ativa uma única saída dentre as 2^N possíveis. Para o caso específico de um decodificador 3 para 8:

- **Entradas:** 3 bits, permitindo $2^3 = 8$ combinações distintas;
- **Saídas:** 8 linhas, onde apenas uma permanece ativa por vez;
- **Enable:** sinal de habilitação que controla o funcionamento do decodificador.

O desenvolvimento foi realizado no software Intel Quartus II, utilizando a técnica de **processos** com estruturas de controle *if-then-else* e *case-when* para implementação sequencial do circuito. A validação foi efetuada através de simulações com arquivos de formas de onda (*University Program VWF*).

2 DESENVOLVIMENTO

2.1 Fundamentos Teóricos

2.1.1 Processos em VHDL

Conforme as Notas de Aula 16, processos são regiões de código **sequencial** em VHDL, onde os comandos são avaliados respeitando a ordem na qual estão localizados no código. Esta característica difere do comportamento padrão da linguagem VHDL, onde os comandos são concorrentes.

A sintaxe básica de um processo inclui uma **lista de sensibilidade**, que especifica os sinais responsáveis por acionar a execução do processo. Para circuitos combinacionais, todos os sinais que afetam as saídas devem estar presentes nesta lista.

2.1.2 Estruturas de Controle Sequenciais

As estruturas `if-then-else` e `case-when` são comandos sequenciais que devem ser utilizados exclusivamente dentro de processos:

- **if-then-else**: utilizada para verificações condicionais, como o estado do sinal *enable*;
- **case-when**: utilizada para seleção múltipla baseada no valor de uma expressão, ideal para implementar a lógica de decodificação.

2.2 Identificação das Variáveis de Entrada e Saída

2.2.1 Variáveis de Entrada

O decodificador possui duas categorias de entradas, conforme descrito na Tabela 1.

Tabela 1 – Definição das variáveis de entrada do decodificador

Variável	Tipo	Bits	Descrição
entr	bit_vector	3	Código binário de entrada (E2, E1, E0)
enable	bit	1	Sinal de habilitação do decodificador

Fonte: Elaborado pelo autor, 2025.

2.2.2 Variáveis de Saída

O decodificador possui um vetor de 8 bits como saída, conforme apresentado na Tabela 2.

Tabela 2 – Definição das variáveis de saída do decodificador

Variável	Tipo	Bits	Descrição
saidas	bit_vector	8	Saídas do decodificador (S7 a S0)

Fonte: Elaborado pelo autor, 2025.

2.3 Tabela-Verdade

A Tabela 3 apresenta o comportamento completo do decodificador 3 para 8 com *enable*. Quando o sinal de habilitação está em nível lógico '0', todas as saídas permanecem desativadas, independentemente do valor das entradas.

Tabela 3 – Tabela-verdade do decodificador 3 para 8 com *enable*

Enable	E2	E1	E0	S7	S6	S5	S4	S3	S2	S1	S0
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Fonte: Elaborado pelo autor, 2025.

Observação: O símbolo "X" indica condição irrelevante (*don't care*), pois quando *enable* = '0', as entradas não influenciam as saídas.

2.4 Código VHDL

O Quadro 1 apresenta o código VHDL desenvolvido utilizando a técnica de processos com estruturas *if-then-else* e *case-when*, conforme metodologia apresentada nas Notas de Aula 16.

Quadro 1 – Código VHDL do decodificador 3 para 8 com *enable*

```

-- =====
-- Decodificador 3 para 8 com Enable
-- Disciplina: BiSuEEA. 512 - Sistemas Embarcados
-- Descrição: Circuito combinacional que recebe um código
--             binário de 3 bits e ativa uma única saída
--             dentre as 8 possíveis.
-- =====

library ieee;
use ieee.std_logic_1164.all;

-- Entradas e saídas
entity decod_3_8 is
port(
  entr    : in  bit_vector(2 downto 0); -- 3 bits de entrada
  saidas  : out bit_vector(7 downto 0); -- 8 saídas
  enable  : in  bit                    -- Sinal de habilitação
);
end decod_3_8;

-- Lógica do circuito
architecture funcionamento of decod_3_8 is
begin
  -- Processo com lista de sensibilidade (entr, enable)
  process(entr, enable)
  begin
    -- Verifica se o decodificador está habilitado
    if enable = '1' then
      -- Estrutura case-when para decodificação
      case entr is
        when "000" => saidas <= "00000001"; -- S0 ativo
        when "001" => saidas <= "00000010"; -- S1 ativo
        when "010" => saidas <= "00000100"; -- S2 ativo
        when "011" => saidas <= "00001000"; -- S3 ativo
        when "100" => saidas <= "00010000"; -- S4 ativo
        when "101" => saidas <= "00100000"; -- S5 ativo
        when "110" => saidas <= "01000000"; -- S6 ativo
        when "111" => saidas <= "10000000"; -- S7 ativo
      end case;
    else
      -- Enable = '0': todas as saídas desativadas
      saidas <= "00000000";
    end if;
  end process;
end funcionamento;

```

Fonte: Elaborado pelo autor, 2025.

2.4.1 Estrutura do Código

O código VHDL está organizado conforme a estrutura padrão apresentada nas Notas de Aula 16:

- a) **Bibliotecas:** inclusão da biblioteca `ieee` e do pacote `std_logic_1164`, garantindo compatibilidade com o padrão IEEE 1076;

- b) **Entidade** (*entity*): declaração das portas de entrada e saída:
 - *entr*: vetor de 3 bits (*bit_vector(2 downto 0)*);
 - *saidas*: vetor de 8 bits (*bit_vector(7 downto 0)*);
 - *enable*: sinal simples do tipo *bit*;
- c) **Arquitetura** (*architecture*): implementação da lógica utilizando:
 - **Processo** com lista de sensibilidade (*entr, enable*);
 - Estrutura *if-then-else* para verificar o sinal *enable*;
 - Estrutura *case-when* para decodificação das 8 combinações.

2.4.2 Justificativa do Uso de Processos

A utilização de processos é necessária para empregar as estruturas *if-then-else* e *case-when*, que são comandos sequenciais e não podem ser utilizados na região concorrente do código VHDL.

A **lista de sensibilidade** (*entr, enable*) garante que o processo seja executado sempre que qualquer um desses sinais sofrer alteração, mantendo o comportamento combinacional correto do circuito.

2.4.3 Cobertura Completa do Case

Como o tipo *bit_vector(2 downto 0)* possui exatamente 8 combinações possíveis e todas estão explicitamente cobertas no *case-when*, não é necessário utilizar a cláusula *when others*.

2.5 Implementação no Quartus II

2.5.1 Geração do Bloco Lógico

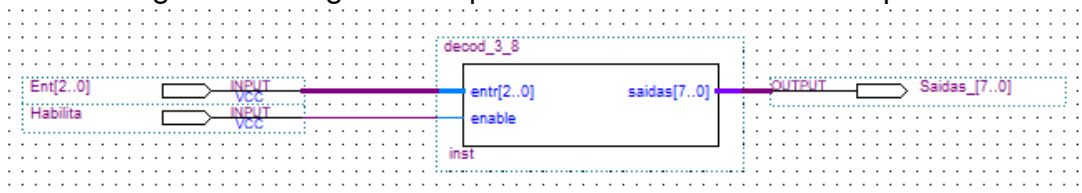
Após a criação do arquivo VHDL, foram executados os seguintes procedimentos:

- a) **Análise sintática**: *Processing* → *Analyze Current File*;
- b) **Geração do símbolo**: *File* → *Create/Update* → *Create Symbol Files for Current File*;
- c) **Compilação**: definição do arquivo esquemático como *Top-Level Entity* e execução da compilação completa.

2.5.2 Diagrama Esquemático

A Figura 1 apresenta o diagrama esquemático construído no Quartus II, contendo o bloco lógico `decod_3_8` gerado a partir do código VHDL.

Figura 1 – Diagrama esquemático do decodificador 3 para 8



Fonte: Elaborado pelo autor, 2025.

Conforme orientações das Notas de Aula 16, foram observados os seguintes cuidados na construção do esquemático:

- Nomenclatura diferenciada:** os nomes dos pinos de entrada/saída do esquemático diferem dos nomes declarados na *entity* VHDL:
 - Entrada: `Ent[2.. 0]` (ao invés de `entr`);
 - Entrada: `Habilita` (ao invés de `enable`);
 - Saída: `Saidas_[7..0]` (ao invés de `saídas`);
- Notação de vetores:** utilização da notação do Quartus II com colchetes `[N.. 0]` para representar barramentos;
- Conexões:** linhas grossas indicam barramentos (múltiplos bits) e linhas finas indicam sinais simples (1 bit).

2.6 Simulação e Validação

2.6.1 Configuração das Formas de Onda

Para validar o funcionamento do decodificador, foi criado um arquivo de simulação (*University Program VWF*) com as seguintes configurações:

- **End Time:** 32 μ s;
- **Grid Size:** 1,28 μ s.

Os sinais de entrada foram configurados com os seguintes períodos e *duty cycle* de 50%:

- entrada `Habilita`: período de 32 μ s (primeira metade desabilitado);
- entrada `Ent[2]`: período de 8 μ s (bit mais significativo);
- entrada `Ent[1]`: período de 4 μ s;
- entrada `Ent[0]`: período de 2 μ s (bit menos significativo).

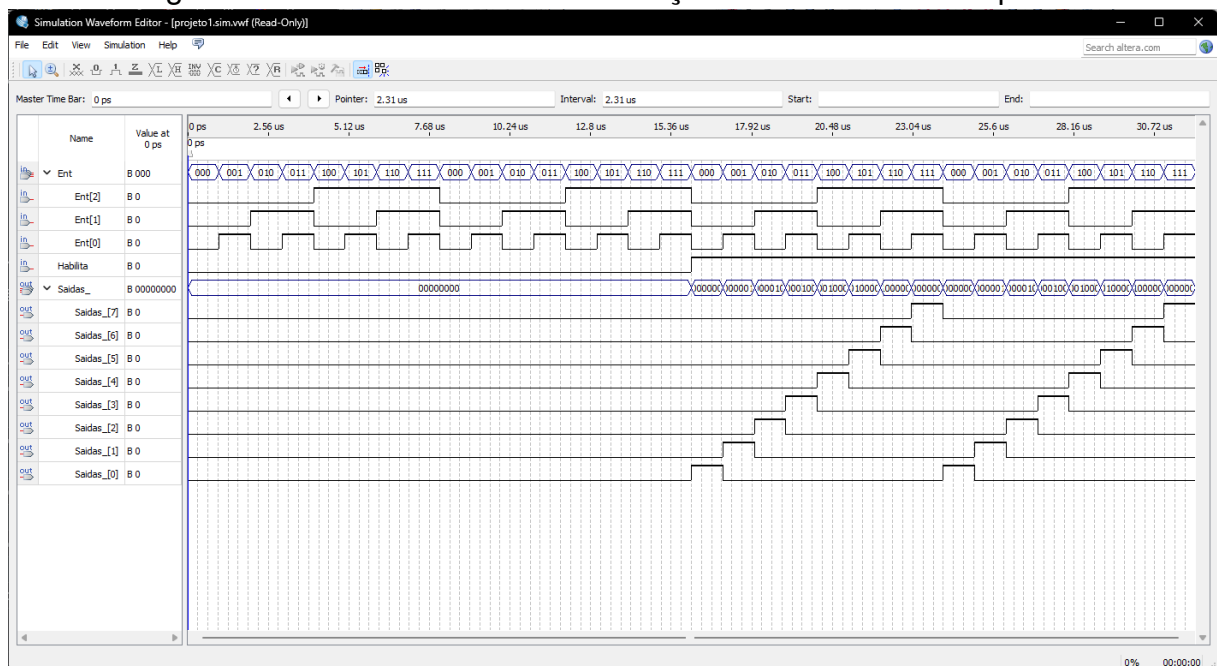
Esta configuração permite verificar o comportamento do decodificador em duas situações distintas:

- 0 a 16 μs : *enable* = '0' (decodificador desabilitado);
- 16 a 32 μs : *enable* = '1' (decodificador habilitado).

2.6.2 Resultados da Simulação

A Figura 2 apresenta as formas de onda resultantes da simulação funcional executada no Quartus II.

Figura 2 – Formas de onda da simulação do decodificador 3 para 8



Fonte: Elaborado pelo autor, 2025.

2.6.3 Análise dos Resultados

Analisando as formas de onda apresentadas na Figura 2, verifica-se que:

- Intervalo 0–16 μs** (*Habilita* = 0): o vetor *Saida_* permanece constantemente em 00000000, independentemente das variações nas entradas *Ent*[2..0]. Este comportamento confirma o funcionamento correto do sinal de habilitação;
- Intervalo 16–32 μs** (*Habilita* = 1): o decodificador opera normalmente, ativando uma única saída para cada combinação de entrada, conforme a codificação *one-hot*.

A Tabela ?? apresenta a correspondência entre as combinações de entrada e as saídas obtidas durante o período de habilitação.

Adicionalmente, a Tabela ?? confirma o comportamento quando o decodificador está desabilitado.

Tabela 4 – Verificação dos resultados da simulação (Enable = 1)

Ent[2.. 0]	Saidas_[7..0] Esperado	Saidas_[7..0] Obtido	Status
000	00000001	00000001	OK
001	00000010	00000010	OK
010	00000100	00000100	OK
011	00001000	00001000	OK
100	00010000	00010000	OK
101	00100000	00100000	OK
110	01000000	01000000	OK
111	10000000	10000000	OK

Fonte: Elaborado pelo autor, 2025.

Tabela 5 – Verificação dos resultados da simulação (Enable = 0)

Ent[2..0]	Saidas_[7..0] Esperado	Saidas_[7..0] Obtido	Status
XXX (qualquer)	00000000	00000000	OK

Fonte: Elaborado pelo autor, 2025.

Os resultados obtidos na simulação correspondem integralmente aos valores esperados da tabela-verdade, confirmando a corretude da implementação do decodificador 3 para 8 com *enable*.

3 CONCLUSÃO

Este trabalho apresentou o projeto e implementação de um decodificador 3 para 8 com entrada de habilitação utilizando a linguagem de descrição de hardware VHDL. O circuito foi desenvolvido empregando a técnica de **processos**, conforme metodologia apresentada nas Notas de Aula 16 da disciplina.

A implementação utilizou as seguintes estruturas de controle sequenciais:

- `if-then-else`: para verificação do sinal de habilitação;
- `case-when`: para decodificação das 8 combinações de entrada.

O uso de processos mostrou-se adequado para implementação de circuitos combinacionais que requerem estruturas de controle sequenciais, permitindo uma descrição clara e organizada da lógica do decodificador. A lista de sensibilidade (`entr`, `enable`) garante que o processo seja executado sempre que houver alteração nos sinais de entrada, mantendo o comportamento combinacional do circuito.

A simulação funcional no Quartus II validou completamente o comportamento do decodificador, demonstrando que:

- a) quando desabilitado (`enable = '0'`), todas as saídas permanecem em nível lógico '0';
- b) quando habilitado (`enable = '1'`), exatamente uma saída é ativada para cada combinação de entrada, implementando corretamente a codificação *one-hot*.

O código VHDL desenvolvido está em conformidade com o padrão IEEE 1076 e é sintetizável em dispositivos FPGA através do Intel Quartus II. A abordagem adotada segue o padrão estabelecido no arquivo de referência `Codigo_decod_02.vhd` fornecido na base de conhecimento da disciplina.